BREAKING MEMORIZATION BARRIERS IN LLM CODE FINE-TUNING VIA INFORMATION BOTTLENECK FOR IMPROVED GENERALIZATION

A PREPRINT

Changsheng Wang[†*] **Xin Chen**[§] **Sijia Liu**[†,‡] **Ke Ding**[§]
[†]The OPTML Lab, Michigan State University
[§]Intel
[‡]IBM Research

ABSTRACT

Adapting pretrained large language models (LLMs) to code domains via supervised fine-tuning (FT) has been commonly used for code generation. However, we identify a previously underappreciated failure mode, the *memorization barrier*, where strong memorization of downstream code data in the base model could trap optimization and prevent the standard FT from effectively acquiring new, generalizable code knowledge. To overcome this barrier, we propose the *information bottleneck (IB)-guided fine-tuning*, termed IB-FT, which applies an IB penalty on hidden representations of the code data to compress spurious, memorized features while preserving task-relevant information. Extensive experiments on two code benchmarks (OriGen and Evol-CodeAlpaca-V1) show that IB-FT substantially alleviates the memorization barrier, improves top-1 performance (Pass@1), and yields far more stable gains under the stricter multi-sample metric Pass@$k^{(m)}$ (a problem counts as solved only if at least m of k samples pass unit tests) compared with conventional FT.

1 Introduction

Code generation sits at the nexus of AI and software engineering, driving rapid advances in both research and industry (Nijkamp et al., 2023; Hou et al., 2024; Thakur et al., 2024; Huynh & Lin, 2025; Li et al., 2022). By automating programming tasks, LLMs can boost developer productivity, lower engineering costs, and increase accessibility, contributions already translate to billions in annual economic value and are expected to grow as adoption widens (Daniotti et al., 2025).

Yet, rather than training models from scratch on specialized code corpora, practitioners typically fine-tune pretrained LLMs on downstream code datasets because this approach is computationally cheaper and more accessible (Roziere et al., 2023; Beckmann et al., 2004; Wang et al., 2022). Nevertheless, adaptation remains challenging: models must acquire both programming syntax and compositional semantics while avoiding overfitting to dataset idiosyncrasies and excessive memorization of pretraining data (Mathews & Nagappan, 2024; Xu et al., 2022; Fakhoury et al., 2024). In this work, we show that code fine-tuning is often brittle: fine-tuned models can exhibit a large gap between greedy decoding (Pass@1) and sampling-based metrics (Pass@k), *i.e.*, a correct program may appear among multiple (k) sampled generations while the one-time generation by Pass@1 remains incorrect; We defer further details to Fig. 2 (Sec. 3). This evidence exposes the limited effectiveness of current code fine-tuning practices and motivates our question:

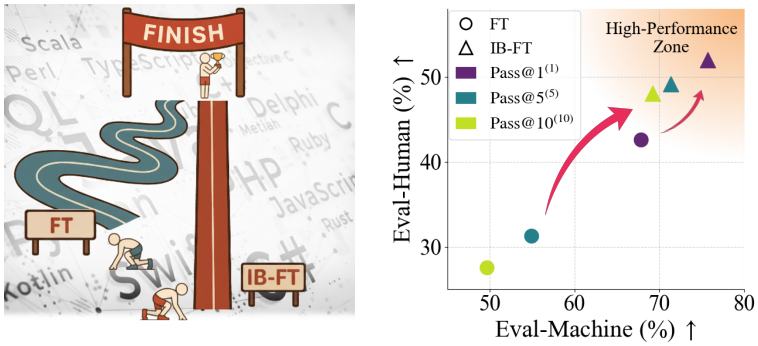
> ***(Q)** What causes the ineffectiveness of LLM code fine-tuning, and how can fine-tuning be modified to reliably improve generalization?*

To tackle **(Q)**, we first investigate why code fine-tuning is not effective to acquire new code knowledge, examining the interaction between fine-tuning data and the pretrained model through the lens of *memorization*. We identify a "***memorization barrier***": the base model already strongly memorizes the fine-tuning code data, trapping optimization in

[*]This work was initiated during an internship at Intel, and part of it was completed at OPTML after the internship.

a region that the standard fine-tuning objective cannot escape. Although prior work has identified memorization as a core challenge for LLM-based code generation (Chen et al., 2025; Yang et al., 2024; Al-Kaswan et al., 2024), most studies treat memorization as a privacy or contamination problem, either by detecting leakage against pretraining or fine-tuning corpora (Carlini et al., 2022; Zeng et al., 2023) or by showing that test-set contamination in pretraining inflates evaluation scores (Deng et al., 2024; Dong et al., 2024; Golchin & Surdeanu, 2025; Wang et al., 2025; Riddell et al., 2024). In contrast, we adopt a cross-setting perspective and show that strong memorization of the fine-tuning dataset already present in the base model (prior to adaptation) creates a "memorization barrier"[2] that traps optimization and prevents effective, generalizable fine-tuning.

To overcome the memorization barrier, we also propose a novel fine-tuning (FT) strategy, termed *IB-regularized fine-tuning* (IB-FT), motivated by the information bottleneck (IB) principle (Tishby & Zaslavsky, 2015; Saxe et al., 2019). IB-FT applies an IB regularizer to hidden representations to compress prediction-irrelevant (spurious) features that arise from memorization bias. By constraining representation capacity and reshaping the data representation distribution, IB-FT reduces the dominance of heavily memorized samples and promotes more uniform learning across the code dataset, improving domain adaptation and generalization to unseen code. Intuitively, consider two runners: one starts early but follows a winding, indirect path (standard FT, which appears ahead by memorization yet is slowed by detours), while the other starts at the official line but runs straight to the finish (IB-FT, which, despite a later start, reaches the goal more directly). See **Fig. 1** for an illustrative schematic and performance comparison.



(a) Code fine-tuning race (b) Fine-tuning on OriGen

Figure 1: Schematic overview of IB-FT versus conventional FT and a representative performance comparison when fine-tuning on OriGen. (a) A code fine-tuning "race": conventional FT (left) appears to advance quickly by memorization but follows a winding, less-generalizable path, whereas IB-FT (right) progresses along a straighter trajectory toward the "winning" generalization. (b) Test-time performance for DeepSeek-Coder-7B-Instruct-v1.5, reported on Eval-Human and Eval-Machine using Pass@$k^{(m)}$ with $m = k \in \{1, 5, 10\}$ (a problem is counted only if all k samples pass). IB-FT consistently attains higher, more stable accuracy and occupies the high-performance region compared to FT.

We summarize our main contributions below:

• We introduce and formalize the memorization barrier problem, showing via systematic analysis that strong pre-existing memorization of fine-tuning examples can critically limit the effectiveness of code fine-tuning.

• We recast fine-tuning through the IB (information bottleneck) lens and propose an IB-based regularizer (IB-FT) that compresses spurious features, alleviates the memorization barrier, and promotes more effective code fine-tuning.

• We present extensive empirical results validating the memorization barrier phenomenon and benchmark IB-FT on two diverse code datasets, OriGen (Cui et al., 2024) and Evol-CodeAlpaca-V1 (Luo et al., 2024), showing that IB-FT consistently improves generalization, including under stricter evaluations such as Pass@1.

2 Related Work

LLM code fine-tuning. Prior work on code fine-tuning has improved model performance via data-side interventions, *e.g.*, synthetic or augmented corpora (Luo et al., 2023; Li et al., 2023), and method-side advances, *e.g.*, multi-task FT, self-alignment, and parameter-efficient tuning (Ma et al., 2023; Wei et al., 2024; Liu et al., 2024a; Zhuo et al., 2024). However, these efforts largely ignore memorization in the training data, preventing the potential of code datasets from being fully exploited.

[2] Although introduced for code generation, the memorization barrier likely extends to other domains where pretrained models already memorize downstream data, thereby hindering effective adaptation.

Information bottleneck (IB) in LLMs. The information bottleneck (IB) principle (Tishby & Zaslavsky, 2015; Shwartz-Ziv & Tishby, 2017; Yang et al., 2025) prescribes that latent representations should retain task-relevant information while discarding irrelevant details. Prior IB-based work has applied this idea to reasoning, calibration, security, and representation analysis, emphasizing compression or mutual-information estimation (Chen et al., 2024; Li et al., 2025; Liu et al., 2024b; Lei et al., 2025). By contrast, to the best of our knowledge, we are the first to apply IB-guided fine-tuning in code generation, specifically to suppress spurious memorized features during adaptation.

3 Preliminaries and Challenges in LLM Code Fine-tuning

In this section, we introduce the preliminaries of LLM code fine-tuning, the primary task under study, and highlight its central challenge—the limited effectiveness of fine-tuning in achieving generalization on code generation tasks. We then present the core problems of our interest, serving as the focus of the subsequent sections.

Preliminaries on LLM code fine-tuning. LLMs are pretrained on massive and heterogeneous corpora, which equip them with broad linguistic and reasoning capabilities. While such pretraining confers strong general abilities, it often falls short in providing the specialized expertise required for code-related tasks, *e.g. reasoning beyond code syntax (Jain et al., 2023), Python code generation (Le et al., 2022), domain-specific adaptation for Verilog code generation Thakur et al. (2024); Zhao et al. (2024)*. Instead of training an LLM from scratch on code-centric corpora, the conventional approach is to adapt a pretrained model through *code fine-tuning*, where specialized code datasets are used to align the model with the target code distribution (Cui et al., 2024; Wei et al., 2025).

Formally, given a pretrained model with parameters $\boldsymbol{\theta}_0$ and a target-domain code dataset $\mathcal{D}_{\text{code}} = \{(x, y)\}$, where x denotes the input (*e.g.*, problem description) and y denotes the target output (*e.g.*, solution code), the fine-tuning objective is to minimize the negative log-likelihood:

$$\underset{\boldsymbol{\theta}}{\text{minimize}} \ \mathbb{E}_{(x,y)\sim\mathcal{D}_{\text{code}}}\big[-\log p_{\boldsymbol{\theta}}(y \mid x)\big], \tag{1}$$

where $\boldsymbol{\theta}$ are the model parameters initialized from $\boldsymbol{\theta}_0$. LLM-based code models drive applications such as code completion, program synthesis, and translation. Their potential to enhance productivity, reduce errors, and democratize programming underscores the need for effective code fine-tuning.

As for Verilog code generation evaluation, *VerilogEval* is commonly used benchmark (Liu et al., 2023), which consists of two complementary subsets. *Eval-Machine* refers to automatic evaluation, where generated code is executed against unit tests or reference outputs (Liu et al., 2023). *Eval-Human*, in contrast, relies on human annotators to assess the quality of generated code in terms of correctness, clarity, and style (Liu et al., 2023). The benchmark uniformly adopts the well-known Pass@k metric, which measures the probability that at least one of the top-k generated samples passes the unit tests (Chen et al., 2021).

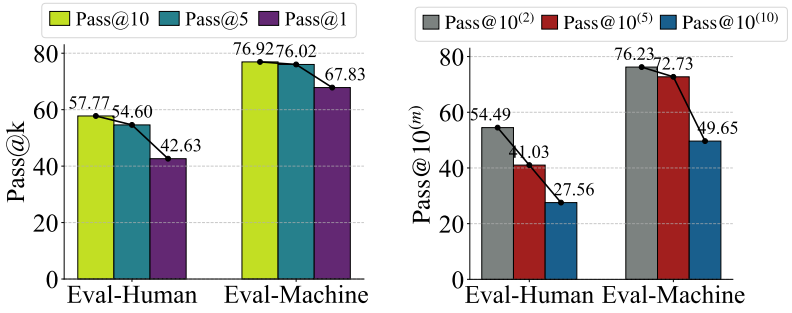


Figure 2: Performance of DeepSeek-Coder-7B-Instruct-v1.5 fine-tuned on the OriGen dataset, evaluated on the Eval-Human and Eval-Machine. (a) Pass@k results for $k \in \{10, 5, 1\}$, measuring the probability that at least one of the top-k generations passes the test cases. (b) Pass@$k^{(m)}$ results with $k = 10$ and $m \in \{2, 5, 10\}$, requiring multiple successful generations among the ten samples to reflect robustness.

An illusion of effectiveness in code fine-tuning. Before our study, it was commonly believed that code fine-tuning delivers acceptable performance, particularly under the *Pass@k* metric, which measures the probability that *at least one* of the top-k generated outputs from an LLM is correct. However, we find that this might be an illusion of effectiveness: (1) under greedy decoding (*i.e.*, $k = 1$), the fine-tuned LLM can experience a sharp accuracy drop, and (2) even for larger k, most generations actually fail despite one occasionally passing.

Fig. 2 validates the above two limitations by fine-tuning the pretrained DeepSeek-Coder-7B-Instruct-v1.5 on the OriGen, with test-time generalization performance evaluated on Eval-Human and Eval-Machine. As shown in Fig. 2(a), the